

Tercera semana

Continuando con las actividades orientadas al desarrollo de un sistema de información que permita la consulta y modificación de datos sobre una hoja de vida de un equipo biomédico, es clave poder conocer las características mismas de la tecnología y determinar por una parte que datos serían relevantes encontrar en una hoja de vida y de otro lado, obtener un diseño agradable al usuario final. En ese orden de ideas, se plantean los siguientes retos:

- Definir cuales son los campos que debería tener una hoja de vida de acuerdo con la tecnología seleccionada.
- Diseñar una interfaz de usuario a partir de la institución que han venido trabajando como punto de inspiración donde se explique cómo funciona el sistema de información.
- Realizar una maquetación propia del modelo de hoja de vida que permita al usuario la manipulación de la información.

Lo anterior lo pueden implementar a manera de prueba en cualquier formato de office, de tal manera que a la hora de desarrollarlo en los formularios propios de VBA sirva como insumo.

Creando Procedimientos en VBA

Los procedimientos son las unidades de código más pequeños que se pueden ejecutar Private, Public y Sub; estos mismos se pueden crear en libros y hojas de Excel. Por lo general se van a trabajar en los Módulos (menú>Insertar>módulo). De acuerdo con lo anterior, antes de crear un procedimiento hay que crear un módulo.

Es importante tener en cuenta que al utilizar un Public Sub se puede utilizar en diferentes módulos o macros; mientras con Private Sub solamente se utiliza sobre el módulo activo.

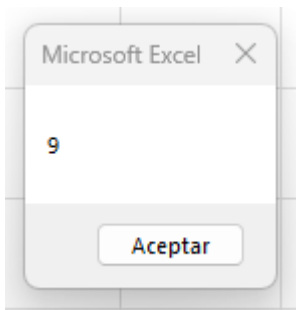
Para ello se deben utilizar los diferentes identificadores de las variables, los cuales no deben tener espacios, caracteres especiales, valores numéricos. No puede haber dos módulos con el mismo identificador.

Un ejemplo:

```
(General)
Private Sub Macro_Privada(Variable1 As Double, Variable2 As Double)
    MsgBox Variable1 + Variable2
End Sub

Public Sub Macro_Publica()
    Macro_Privada 3, 6
End Sub
```

Como resultado se obtiene:



Procedimientos con parámetros

Se pueden crear procedimientos con uno o más parámetros declarados dentro del paréntesis ubicado en el identificador de procedimiento. Se puede declarar o no el tipo de una variable al principio del procedimiento; sin embargo, el no hacerlo conlleva a utilizar una gran cantidad de memoria en el momento de la ejecución. A continuación, un ejemplo:

(General)	SumaNumeros
<pre>Sub SumaNumeros(Variable1 As Double, Variable2 As Double, Variable3 As Double) MsgBox "La suma de las variables es:" & Variable1 + Variable2 + Variable3 End Sub Sub TotalSumaNumeros() Call SumaNumeros(1, 2, 3) End Sub Sub Multiplicadox2(Valor As Variant, Multiplicadox As Double) Debug.Print Valor * Multiplicadox End Sub Sub ResultadoMultiplicacion() Multiplicadox2 2, 2 End Sub</pre>	

Procedimientos con parámetros opcionales

No es necesario proporcionarle argumentos al llamar al procedimiento; en ese sentido, es necesario colocar la palabra reservada “opcional” y debe ir antes del identificador del parámetro. Por ejemplo:

```
Sub ValorIva(ValorProducto As Double, Optional Iva As Double)
    If Iva > 0 Then
        Debug.Print "El Iva total es: " & ValorProducto * Iva
    Else
        Debug.Print "Sin Iva"
    End If
End Sub

Sub CalculoImpuesto()
    ValorIva 200000, 0.19
End Sub
```

Creando una Función en VBA

Como ya es claro, una rutina permite crear una serie de instrucciones en una hoja de cálculo. Sin embargo, existen las funciones (Function) que a diferencia de los procedimientos fueron diseñadas para devolver un valor como resultado; este valor se puede retornar o en caso contrario dejar vacío tipo “variant”.

De igual manera las funciones cuentan con dos ámbitos las cuales pueden ser Privadas o públicas.

Otra característica es que las funciones se pueden utilizar desde las hojas de cálculo.

Un ejemplo. Se tiene un listado de equipos biomédicos y su valor de alquiler o comodato; se desea calcular el valor de Iva y de paso obtener la lista completa de los equipos en general. Para ello se implementan dos funciones que calcule el valor total y otra función para que se genere la lista.

	A	B
1	Comodato	Valor arrendamiento
2	bomba infusion	\$ 55.000,0
3	monitor	\$ 77.000,0
4	succionador	\$ 45.000,0
5	equipo organos	\$ 40.000,0

Primera función

(General)

ConcatenarEquipos

```

Function Arrendamiento(ValorArrendamiento As Range, Optional Intereses As Boolean) As Double
    Dim Valor As Double

    For i = 1 To ValorArrendamiento.Count
        Valor = Valor + ValorArrendamiento(i, 1)
    Next i

    If Intereses = True Then
        Arrendamiento = Valor + (Valor * 0.19)
    Else
        Arrendamiento = Valor
    End If
End Function

```

Como resultado:

B7

fx

=Arrendamiento(B2:B5;VERDADERO)

	A	B	C	D
1	Comodato	Valor arrendamiento		
2	bomba infusion	\$ 55.000,0		
3	monitor	\$ 77.000,0		
4	succionador	\$ 45.000,0		
5	equipo organos	\$ 40.000,0		
6				
7	Total valor	\$ 258.230,0		

Segunda función:

```

Function ConcatenarEquipos(ValorUnir As Range, Optional Delimitador As String) As String
    Dim Equipos As String

    For i = 1 To ValorUnir.Count
        If i = 1 Then
            Equipos = ValorUnir(i, 1)
        Else
            Equipos = Equipos & Delimitador & ValorUnir(i, 1)
        End If
    Next i

    ConcatenarEquipos = Equipos
End Function

```

Como resultado:

B9					
	A	B	C	D	E
1	Comodato	Valor arrendamiento			
2	bomba infusion	\$ 55.000,0			
3	monitor	\$ 77.000,0			
4	succionador	\$ 45.000,0			
5	equipo organos	\$ 40.000,0			
6					
7	Total valor	\$ 258.230,0			
8					
9	Lista de equipos	bomba infusion,monitor,succionador,equipo organos			
10					

Diferencias entre parámetros y argumentos

Los parámetros representan un valor que en el procedimiento o función espera que se le pase al invocarlo o llamarlo y estos se definen dentro de los paréntesis que se encuentran justo enseguida del procedimiento.

Cuando se llama a un procedimiento o función y tienen uno o más parámetros, se pueden pasar dichos argumentos con el nombre del parámetro seguido de dos puntos y el valor que se le va a asignar a dicho parámetro. A continuación, un ejemplo:

```
Sub Suma(Valor1 As Double, Valor2 As Double)
    MsgBox Valor1 + Valor2
End Sub
```

```
Sub PasarPorPosicion()
```

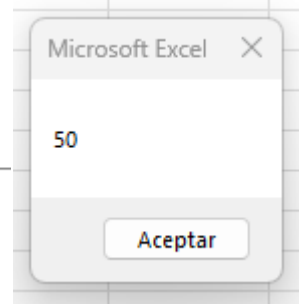
```
    Call Suma(30, 20)
    Suma 30, 20
```

```
End Sub
```

```
Sub PasarPorNombre()
```

```
    Suma Valor1:=30, Valor2:=20
```

```
End Sub
```



Diferencias entre ByVal y ByRef

Existen dos alternativas para pasar argumentos por valor haciendo uso de la palabra ByVal donde el valor cambia en la funcion más no en el procedimiento.

```
Sub Procedimiento()
    Dim ValorProcedimiento As Double

    ValorProcedimiento = 500
    ValorMasDiez ValorProcedimiento
    MsgBox ValorProcedimiento

End Sub
```

```
Sub ValorMasDiez(ByVal Valor As Double)
    Valor = Valor + 10
End Sub
```

⇒

Locales

/BAProject.Módulo1.ValorMasDiez

Expresión	Valor	Tipo
Módulo1		Módulo1/Módulo1
Valor	510	Double

Función Split

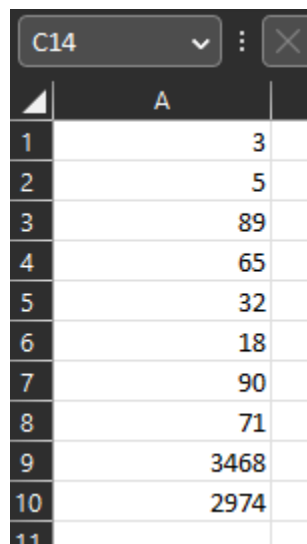
La función Split permite separar caracteres de un texto en particular o en dado caso separar valores delimitados por coma (,) o punto (.).

A continuación, un ejemplo sobre como separar un vector de números:

```
(General)
Sub Ejemplo_Split()
    Dim FilaNumeros() As String
    Dim CadenaNumeros As String

    CadenaNumeros = "3,5,89,65,32,18,90,71,3468,2974"
    FilaNumeros = Split(CadenaNumeros, ",")
    'UBound es el limite superior hasta donde va a recorrer el vector
    For i = 0 To UBound(FilaNumeros)
        'se elige la columna
        Range("a" & i + 1).Value = FilaNumeros(i)
    Next i
End Sub
```

Como resultado se obtiene en la hoja de Excel:



	A
1	3
2	5
3	89
4	65
5	32
6	18
7	90
8	71
9	3468
10	2974
11	

Función Join

Esta función permite unir los datos de un vector en particular separados por un limitador proporcionado en la misma función.

Un ejemplo:

```

Sub Ejemplo_Join()
    Dim ListaAsignaturas(1 To 5) As String

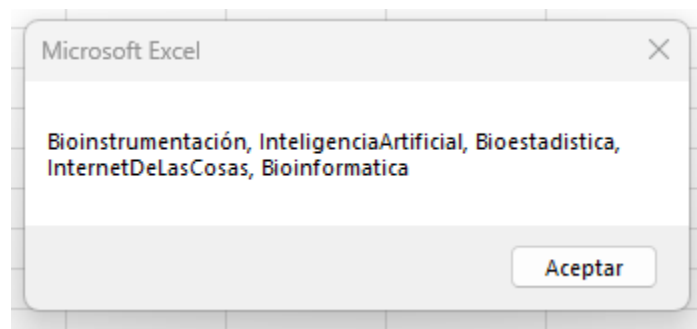
    ListaAsignaturas(1) = "Bioinstrumentación"
    ListaAsignaturas(2) = "InteligenciaArtificial"
    ListaAsignaturas(3) = "Bioestadística"
    ListaAsignaturas(4) = "InternetDeLasCosas"
    ListaAsignaturas(5) = "Bioinformática"

    MsgBox Join(ListaAsignaturas, "," + " ")

End Sub

```

Como resultado se obtiene:



Función Filter

Permite encontrar subconjuntos de elementos que se encuentran en un vector, permitiendo aplicar diferentes criterios para obtener dichos elementos; esta función fue desarrollada para ser utilizada en los arreglos o vectores. Un ejemplo:

```

Sub Ejemplo_Filter()
    Dim ListaAsignaturas(5) As String
    Dim ResultFiltro() As String

    ListaAsignaturas(1) = "Bioinstrumentación"
    ListaAsignaturas(2) = "InteligenciaArtificial"
    ListaAsignaturas(3) = "Bioestadística"
    ListaAsignaturas(4) = "InternetDeLasCosas"
    ListaAsignaturas(5) = "Bioinformática"
    'se asigna la l para buscar las palabras que contengan esta letra
    ResultFiltro = Filter(ListaAsignaturas, "l", True, vbTextCompare)
    MsgBox Join(ResultFiltro, ",")

End Sub

```


Vectores dinámicos

```
(General)

Sub Ejemplo_Vectores()
    Dim Vectores() As Double
    Dim VectorLeido As Double
    Dim Contador As Integer

    Contador = 1
    Do
        VectorLeido = InputBox("Digite un numero")
        If VectorLeido <> 0 Then
            'ReDim ayuda a preservar o dejar almacenado los datos
            ReDim Preserve Vectores(1 To Contador)
            Vectores(Contador) = VectorLeido
            Contador = Contador + 1
        End If

        Loop While VectorLeido <> 0
        For i = 1 To Contador - 1
            Range("a" & i).Value = Vectores(i)
        Next i

        Erase Vectores
        MsgBox "secuencia terminada"

    End Sub
```

Como resultado se obtiene los datos en la hoja de Excel:

D12			✕	✓	<i>fx</i>
	A	B			
1		4			
2		5			
3		7			
4		8			
5		23			
6		56			
7		78			
8		40			

Objeto de aplicación

Los objetos son representados como cosas en Excel y sus características son conocidas como propiedades, las cuales permiten definir al objeto y finalmente, los métodos son las acciones que cada uno de ellos (objetos) puede realizar.

Vale la pena resaltar que a la hora de trabajar con VBA se pueden manejar múltiples objetos que pueden ejecutar diferentes instrucciones adecuadamente; si embargo, el objeto de aplicación esta en un nivel más alto de la jerarquía del modelo de objetos de Excel, ya que permite el acceso a opciones y configuraciones a nivel de la aplicación.

De acuerdo con lo anterior, es frecuente encontrar instrucciones que comiencen especificando el objeto de aplicación. Un ejemplo de ello:

`Application.StatusBar`

`Application.DisplayAlerts`

`Application.InputBox`

Objeto Workbook

Los objetos Workbook representan a los libros de Excel y pertenecen a la aplicación de Excel como tal. Este objeto puede contener desde 1 hasta 255 hojas de cálculo y otros objetos más. En el editor de VBA cada libro representa un proyecto diferente. Por otra parte, este objeto al igual que `Application`, cuenta con algunas propiedades y eventos de los cuales solamente se abordarán los más importantes ya que existen bastantes de ellos ya que abarcan una gran cantidad de eventos, métodos y propiedades

Propiedades y métodos de un formulario

Para esta parte es clave tener habilitada la ventana de propiedades, la cual puede activarla en el menú > Ver > Ventana propiedades.

La primera propiedad para abordar se conoce como `Name`, la cual define el nombre del objeto para luego ser invocado/llamado desde VBA y de esta manera, poder cambiar una propiedad o invocar un método de este.

Vale la pena resaltar que esta propiedad lo tiene cada uno de los controles. En ese sentido, la recomendación es definir en la etapa de diseño el nombre de los botones o formularios, ya que una vez definidos y al cambiarlos de nombre más adelante, puede generar errores.

Otra propiedad es `Caption` que permite asignarle un texto cualquiera en la parte superior del formulario, a manera de título.

Otra propiedad es el `BackColor` y `BorderStyle` que permite cambiar la apariencia del formulario; en este caso se recomienda utilizar la paleta de colores que se encuentra en esta opción.

Para ampliar más sobre el uso de otras propiedades, puede consultar el siguiente link:

<https://learn.microsoft.com/es-es/office/vba/language/reference/user-interface-help/properties-microsoft-forms>

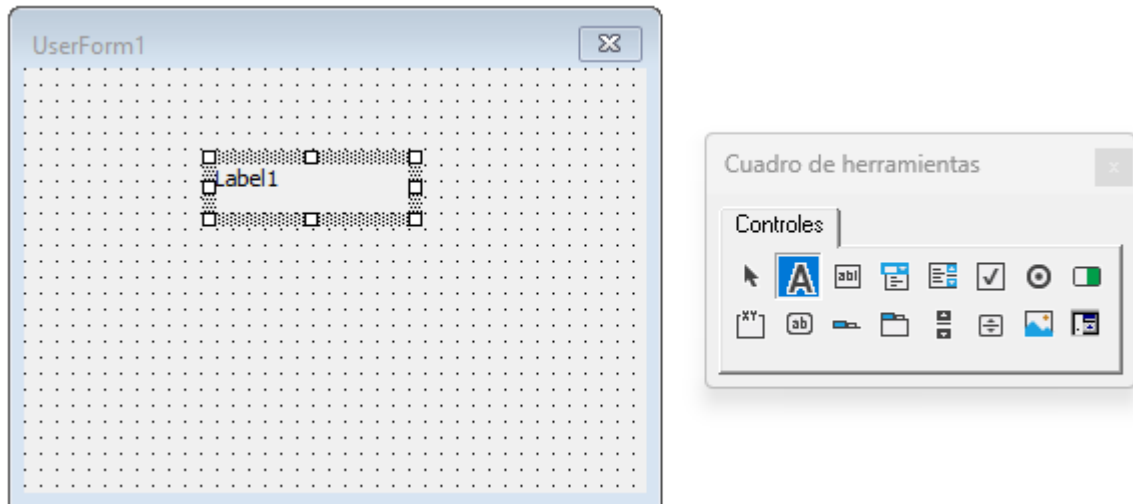
Controles principales y abreviaturas

Cada control debe tener un nombre, por medio del cual se hace referencia a dicho objeto. De hecho, VBA asigna automáticamente un nombre por defecto una vez se vayan duplicando los formularios u objetos como tal. Razón por la cual se presenta la siguiente tabla, el nombre, abreviatura y una sencilla descripción sobre el funcionamiento de cada una de ellas. Lo anterior va de la mano con el nombre que haga referencia a dicho control y que por supuesto debe contar con las mismas condiciones con las que se declara una variable.

Nombre en VBA	significado	Abreviatura	Descripción
UsrrForm	Formulario	Form	Formulario que contiene un conjunto de controles
textBox	Cuadro de texto	Tbx	Control usado para mostrar o editar textos. Con la particularidad de no tener formato
Label	Etiqueta	Lbl	Permite especificar un texto o breves instrucciones en el formulario
Frame	Cuadro de grupo	Frm	Agrupar controles relacionados en una unidad visual. Por lo general, se agrupan botones de opción, casillas de verificación.
Commandbutton	Boton	Cbn	Permite ejecutar una macro al momento de hacer clic
CheckBox	Casilla de verificación	Chx	Permite la selección o no de una opción
OptionButton	Boton de opción	Opb	Permite una única selección dentro de un conjunto de opciones
ListBox	Cuadro de lista	Lbx	Presenta una lista de uno o más elementos de texto para mostrar varias opciones al usuario
Combobox	Cuadro combinado	Cbx	Presenta una lista de valores de los cuales se puede elegir una sola o múltiples opciones, dependiendo la configuración inicial
Scrollbar	Barra de desplazamiento	Scb	Se desplaza por un intervalo de tiempo de valores cuando el usuario hace clic en las flechas de desplazamiento.
Spinbutton	Control de numero	Spb	Aumenta o disminuye el valor, como por ejemplo el incremento numérico de hora, fecha entre otras

Control Label

Permite ingresar texto al UserForm. Basta con hacer clic dentro del mismo para que se habilite la caja de herramientas. Una vez activa, seleccione A (etiqueta) para poder ingresar un texto.



Al ejecutarlo, aparecerá...



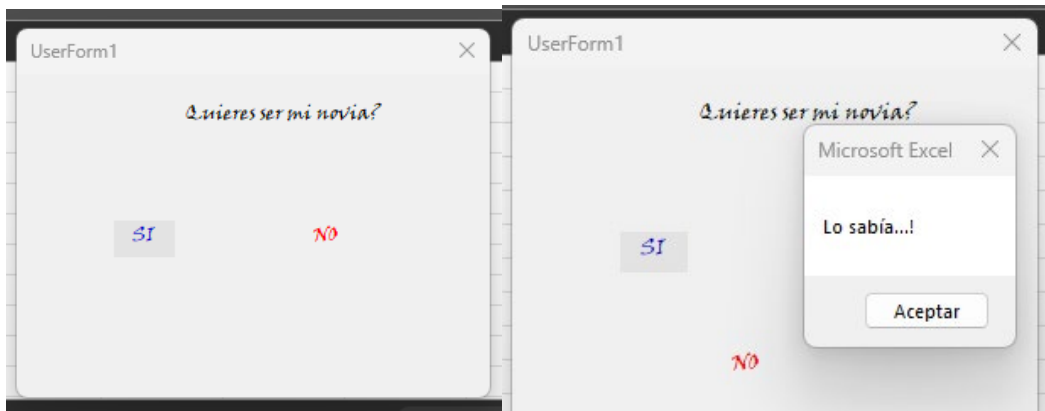
Otras propiedades a tener en cuenta son: Font, ForeColor, BackColor, Color y Visible.

Un ejemplo:

```
Private Sub Lbl_No_MouseMove(ByVal Button As Integer, ByVal Shift As Integer, ByVal X As Single, ByVal Y As Single)
    'se define la margen superior maximo
    Lbl_No.Top = WorksheetFunction.RandBetween(1, Me.Height - 40)
    'se define la margen inferior maxima
    Lbl_No.Left = WorksheetFunction.RandBetween(1, Me.Width - 40)
End Sub

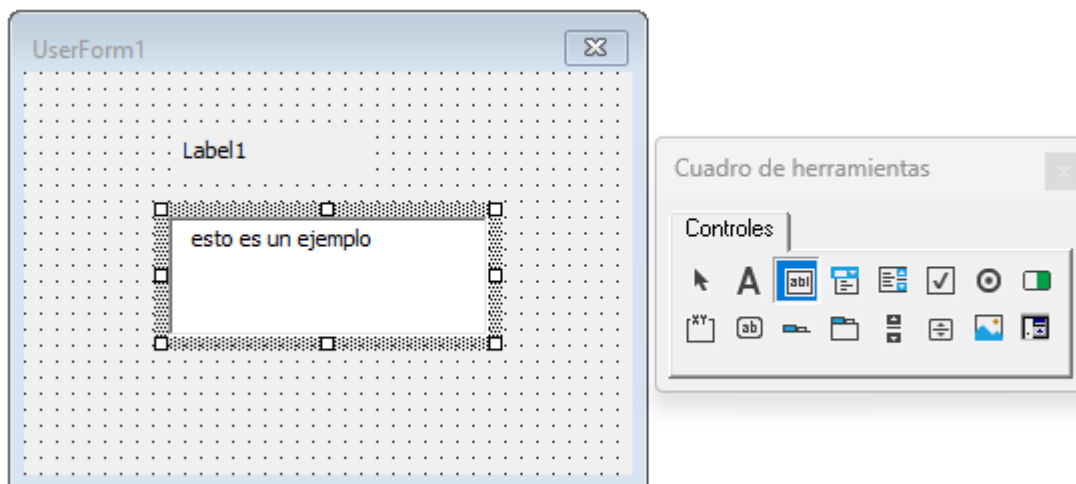
Private Sub Lbl_Si_Click()
    MsgBox ("Lo sabia...!")
End Sub
```

Como resultado:



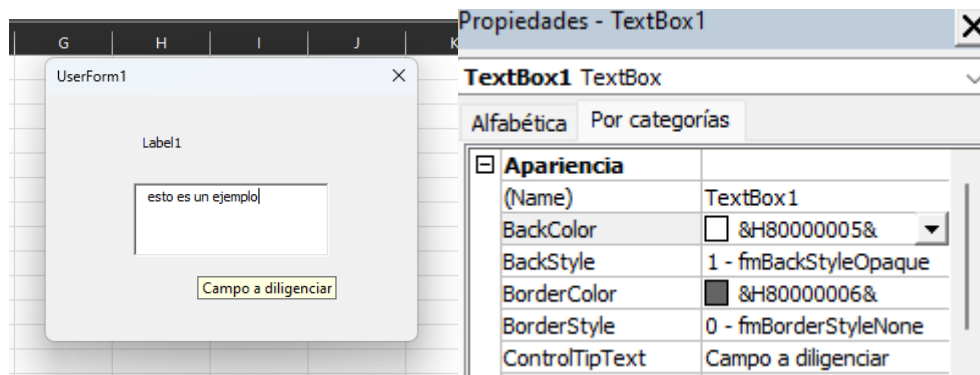
Control TextBox

Es usado para mostrar o editar textos. La manera de utilizarlo en el formulario es muy sencillo; basta con hacer clic en el formulario y en la caja de herramientas hacer clic en “cuadro de texto”.



Sus propiedades son muy parecidas a las utilizadas en los formularios, como es el caso de alineación de texto (TextAling), tamaño de la caja (AutoSize), bloquear escritura (Locked) entre otros.

Un ejemplo de ello, es el uso de la propiedad ControlTipText que muetsra un mensaje cada vez que se pasa el mouse por enciama de la caja de texto.



Formulario de autenticación

Este es un ejemplo básico del funcionamiento de un sistema simple de autenticación.

En una hoja de Excel ingrese los siguientes datos:

usuario	contrasena
pedro	20220022
antonio	20230025
jefferson	2023123
oscar	2023456
julian	123456
juan	987654
fernando	852963
hernan	654321
alfredo	2023321

En primer lugar, implementa el siguiente Form (3 etiquetas y 2 cuadros de texto y 1 botón de comando):



Después de implementado, configura de acuerdo con el siguiente ejemplo:

```

Cbn_Aceptar Click
Private Sub Cbn_Aceptar_Click()
    Dim Mensaje As String

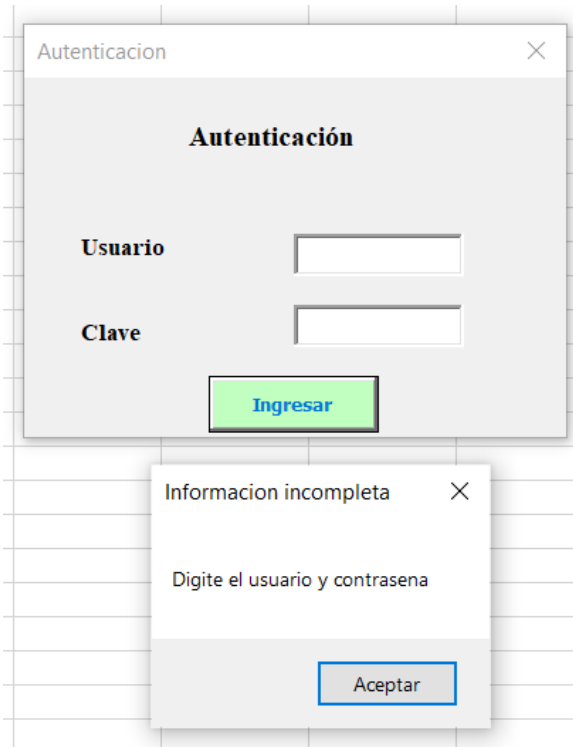
    If Tbx_Usuario.Value = "" Or Tbx_Clave.Value = "" Then
        MsgBox "Digite el usuario y contraseña", vbInformation, "Informacion incompleta"
    Else
        For i = 2 To [a65536].End(xlUp).Row
            If Tbx_Usuario.Value = Range("a" & i).Value Then
                If Tbx_Clave.Value = CStr(Range("b" & i).Value) Then
                    Mensaje = "Bienvenido al curso"
                    Me.Hide
                Else
                    Mensaje = "contraseña invalida"
                End If
                Exit For
            Else
                Mensaje = "Usuario no valido"
            End If
        Next i

        MsgBox Mensaje
    End If
End Sub

```

Prueba los resultados:

Si no se ingresan datos



AL ingresar algún dato erróneo:

